

CSE 4208: Computer Graphics Laboratory

3D Hover Bus City Simulation

By

Md Mofazzal Hosen

Roll: 2007074



Instructors:

Dr. Sk. Md. Masudul Ahsan
Professor

Md. Tajmilur Rahman
Lecturer

Department of Computer Science and Engineering
Khulna University of Engineering & Technology
Khulna 9203, Bangladesh
April 2026

Contents

1	Objectives	2
2	Introduction	2
3	Object Demonstration	4
3.1	Chase Camera View (3rd Person)	4
3.2	Interior Camera View (1st Person)	4
3.3	City Environment with Buildings and Road	5
3.4	Fractal Forest	5
3.5	Ring Checkpoints and HUD Scoreboard	6
3.6	Curve Objects: Bezier Vases, Spline Lamps	7
3.7	Skybox	8
4	Key Functionality	8
4.1	Driving Controls	8
4.2	Camera Controls	8
4.3	Lighting Controls	9
4.4	Bus Interactive Features	9
5	Theory and Pseudocode	9
5.1	OpenGL 3.3 and the Programmable Pipeline	9
5.2	Hierarchical Modeling and Primitives	10
5.3	Custom lookAt Function	10
5.4	Camera System	11
5.5	Lighting and Shading	12
5.6	Texture Mapping	12
5.7	Bezier, Spline, and Ruled Surfaces	13
5.8	Menger Sponge Fractal	13
5.9	Fractal Forest (GPU-Instanced)	14
5.10	Collision Detection	15
5.11	Gamification and Scoring	15
5.12	Skybox Rendering	15
5.13	Rendering Pipeline	15
6	Conclusion	16
7	References	16

1 Objectives

- To create a visually rich 3D hover bus simulation within a procedurally generated infinite city environment using Modern OpenGL 3.3.
- To construct a detailed bus model using hierarchical modeling with primitive shapes (Cube, Cylinder, Torus) built from parametric triangle representations.
- To implement multiple camera systems: Chase Camera (3rd person), Interior Camera (1st person), and Free Camera with orbit functionality.
- To implement Pitch, Yaw, and Roll rotations for the camera along with a custom `lookAt` function.
- To simulate a driving experience with acceleration, deceleration, steering, and vertical hover control (altitude gain/loss).
- To implement multiple lighting models including Directional Light, Point Lights (4), and Spotlight with independent toggle controls.
- To use the Phong shading model (fragment-level) and Gouraud shading model (vertex-level) for realistic illumination with toggleable ambient, diffuse, and specular components.
- To apply texture mapping with multiple modes: pure texture, vertex-blended (Gouraud), fragment-blended (Phong), and multi-texture blending with configurable wrap and filter modes.
- To implement advanced geometric objects using Bezier surfaces of revolution, Catmull-Rom spline surfaces, and Ruled surfaces.
- To create a Menger Sponge fractal (3 iterations, 8000 sub-cubes) rendered via GPU instancing for collectible game elements.
- To implement a recursive fractal forest with textured bark and alpha-cutout leaf billboards, rendered with GPU instancing.
- To build a gamification system with ring checkpoints (Torus and PolygonRing shapes), fractal collectibles, collision detection, and a real-time HUD scoreboard.
- To implement a skybox using cubemap textures for immersive environment rendering.
- To gain hands-on experience in OpenGL 3.3 programmable pipeline, focusing on real-time rendering, GPU instancing, and optimization.

2 Introduction

This project presents a comprehensive 3D hover bus simulation built with Modern OpenGL 3.3. The application places the user in control of a futuristic hover bus navigating through a procedurally generated infinite city environment, complete with buildings, street lamps, vases, canopies, fractal trees, and floating collectibles.

The simulation leverages the OpenGL 3.3 programmable pipeline, utilizing custom vertex and fragment shaders to achieve realistic lighting, shading, and texture mapping. The project employs hierarchical modeling, where the bus is constructed entirely from primitive shapes—Cubes, Cylinders, and Tori—that are transformed via matrix operations to form a detailed vehicle model with interactive doors, a rotating fan, wings, jet engine flames, and hover pads.

The city environment is generated procedurally using deterministic hashing, ensuring that the world is infinite and consistent. Buildings of various types (stacked cubes, tall towers, and cone-topped cylindrical towers) line both sides of a textured road. Bezier surface vases, spline-curve street lamps, and ruled-surface canopies add variety to the streetscape. A fractal forest of recursively branching trees with textured bark and alpha-cutout leaves borders the outskirts.

User interactivity is central to the experience. The driving system supports acceleration, deceleration, steering, and vertical hover control. Three camera modes—Chase, Interior, and Free—allow exploration from multiple perspectives. A gamification layer adds ring checkpoints and floating Menger Sponge fractals as collectibles, with AABB collision detection against buildings penalizing careless driving.

Advanced rendering techniques include GPU-instanced drawing for the fractal forest and Menger Sponges, cubemap skybox rendering, multi-texture blending with spatial transitions, and a bitmap-font HUD system. The combination of these elements creates an engaging, visually appealing, and technically comprehensive demonstration of Modern OpenGL capabilities.

3 Object Demonstration

3.1 Chase Camera View (3rd Person)



Figure 3.1: Chase Camera View — 3rd person perspective behind the hover bus

3.2 Interior Camera View (1st Person)

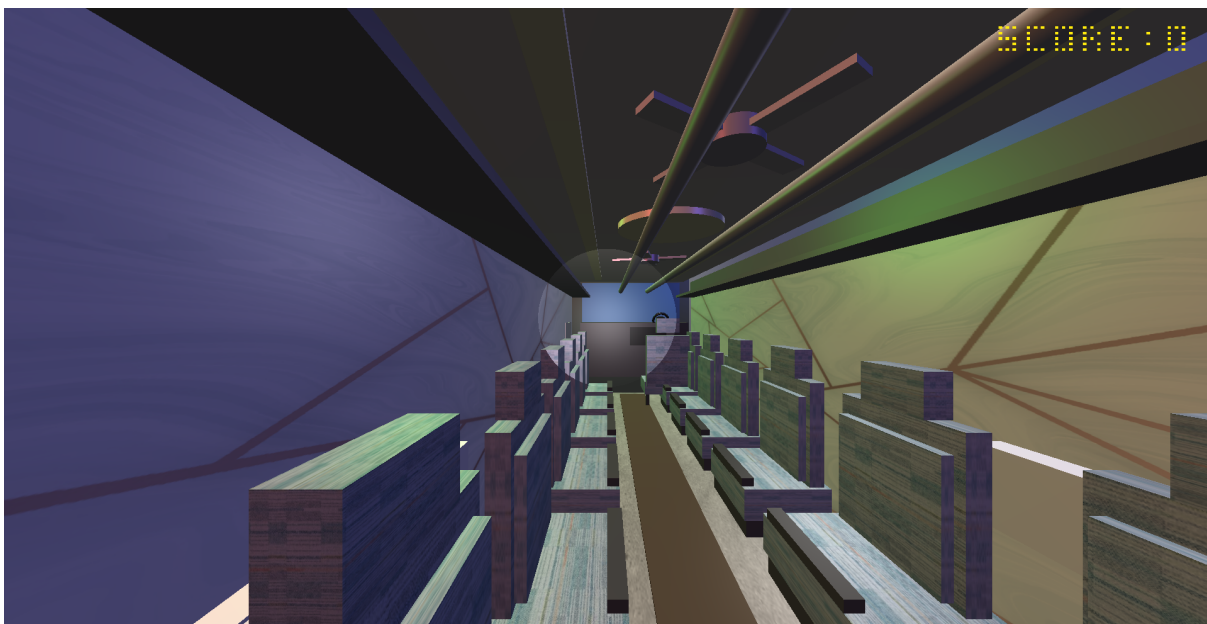


Figure 3.2: Interior Camera View

3.3 City Environment with Buildings and Road

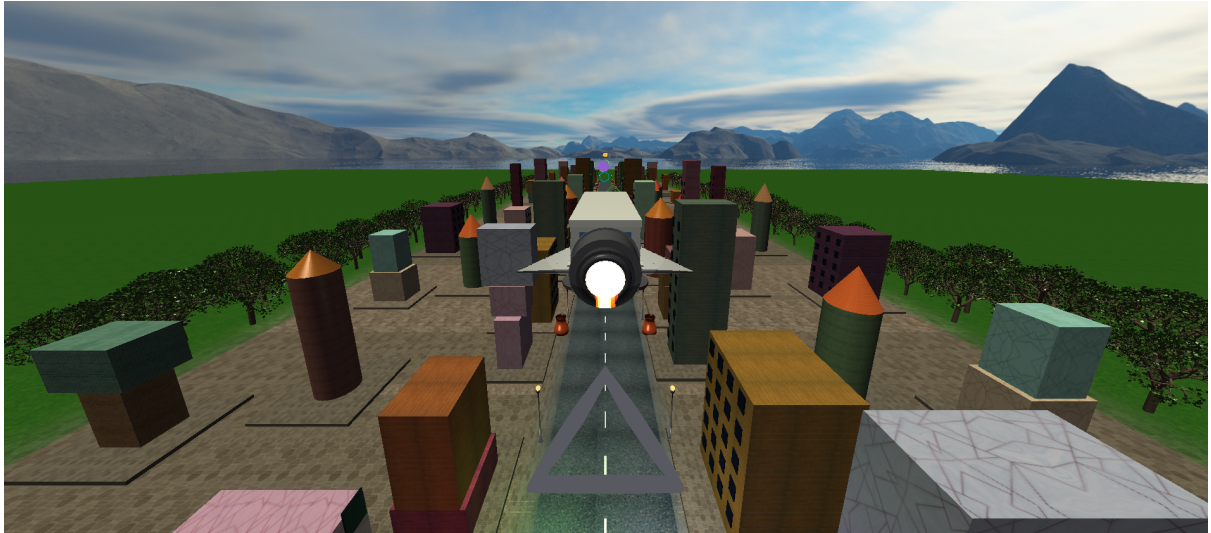


Figure 3.3: Procedural City Environment with varied building types and textured road

3.4 Fractal Forest



Figure 3.5: Fractal Forest (GPU-instanced trees)

3.5 Ring Checkpoints and HUD Scoreboard



Figure 3.6: Ring Checkpoints (varied polygon shapes) and HUD Score Display

3.6 Curve Objects: Bezier Vases, Spline Lamps



Figure 3.7: Bezier Vases, Spline Street Lamps

3.7 Skybox



Figure 3.8: Skybox rendering

4 Key Functionality

4.1 Driving Controls

Key	Description
W	Accelerate the bus forward
S	Accelerate the bus backward (reverse)
A	Steer left
D	Steer right
SPACE	Increase altitude (hover up)
LEFT_CONTROL	Decrease altitude (hover down)

Table 1: Driving Controls

4.2 Camera Controls

Key	Description
V	Cycle camera mode (Chase → Interior → Free)
M	Toggle mouse capture for FPS-style look-around
K	Toggle between driving mode and free camera mode
Arrow Keys	Move free camera (Up/Down/Left/Right)
F (hold)	Orbit camera around the bus
Scroll Wheel	Zoom in/out (adjust FOV)

Table 2: Camera Controls

4.3 Lighting Controls

Key	Description
1	Toggle Directional Light
2	Toggle Point Lights
3	Toggle Spot Light
4	Toggle Emissive Light (flames, glows)
5	Toggle Ambient component
6	Toggle Diffuse component
7	Toggle Specular component

Table 3: Lighting Controls

4.4 Bus Interactive Features

Key	Description
B	Toggle front door open/close
G	Toggle fan spinning
L	Toggle bus interior light
N	Toggle wings ON/OFF
ESC	Close the application
TAB	Print full status to console

Table 4: Bus Interactive Features

5 Theory and Pseudocode

5.1 OpenGL 3.3 and the Programmable Pipeline

OpenGL is a cross-platform API for rendering 2D and 3D graphics. In OpenGL 3.3, the fixed-function pipeline is replaced with a programmable pipeline, providing fine-grained control over rendering. The pipeline is built around shaders written in GLSL (OpenGL Shading Language):

- **Vertex Shaders:** Handle transformations and vertex-level operations. In this project, the vertex shader also computes Gouraud lighting when `textureMode == 2`, and applies per-instance transforms for GPU-instanced rendering of fractal objects.
- **Fragment Shaders:** Handle per-pixel operations including Phong lighting, texture sampling, multi-texture blending, alpha-test cutout for leaf billboards, and emissive rendering for flames and glows.

5.2 Hierarchical Modeling and Primitives

The bus is constructed using hierarchical modeling, where complex objects are built from simple primitives (Cube, Cylinder, Torus) through parent-child transformation chains. Each primitive is built from parametric triangle representations:

- **Cube:** 36 vertices ($6 \text{ faces} \times 2 \text{ triangles} \times 3 \text{ vertices}$), each with position, normal, and texture coordinates.
- **Cylinder:** Parametric generation with configurable sector count, producing side faces, top cap, and bottom cap from triangle strips.
- **Torus:** Parametric generation with configurable major radius, minor radius, main segments, and ring segments.
- **Sphere:** UV sphere with configurable stacks and sectors for the football (texture-mapped sphere).
- **Cone:** Parametric generation similar to cylinder but tapering to a point.

Pseudocode: Hierarchical Bus Construction

Listing 1: Hierarchical Bus Drawing

```
1 FUNCTION drawBus(shader, parentTransform):
2     // Body
3     model = parentTransform * translate(0, 0.5, 0)
4           * scale(10, 3, 3)
5     drawCube(shader, model, bodyColor, busBodyTexture)
6
7     // Wheels (4x) - relative to body
8     FOR each wheel position:
9         wheelModel = parentTransform * translate(wx, wy, wz)
10        * scale(r, r, r)
11        drawTorus(shader, wheelModel, wheelColor)
12
13    // Doors - animated rotation
14    doorModel = parentTransform * translate(doorPos)
15              * rotate(doorAngle, Y_AXIS)
16              * scale(doorSize)
17    drawCube(shader, doorModel, doorColor)
18
19    // Jet Engine - emissive flame
20    setEmissive(true)
21    flameModel = parentTransform * translate(jetPos)
22              * scale(flameSize * flicker)
23    drawCylinder(shader, flameModel, flameColor)
24    setEmissive(false)
```

5.3 Custom lookAt Function

A custom `myLookAt` function replaces `glm::lookAt` to demonstrate understanding of the view matrix construction:

Listing 2: Custom lookAt Implementation

```

1 FUNCTION myLookAt(eye, center, up):
2     f = normalize(center - eye)      // Forward vector
3     s = normalize(cross(f, up))      // Right vector
4     u = cross(s, f)                 // True up vector
5
6     result = identity_matrix
7     // Rotation part
8     result[0][0] = s.x; result[1][0] = s.y; result[2][0] = s.z
9     result[0][1] = u.x; result[1][1] = u.y; result[2][1] = u.z
10    result[0][2] = -f.x; result[1][2] = -f.y; result[2][2] = -f.z
11    // Translation part
12    result[3][0] = -dot(s, eye)
13    result[3][1] = -dot(u, eye)
14    result[3][2] = dot(f, eye)
15    RETURN result

```

5.4 Camera System

Three camera modes are implemented, all using the custom myLookAt:

1. **Chase Camera (3rd Person):** Positioned behind and above the bus, automatically tracking the bus orientation. The camera offset is computed from the bus forward vector.
2. **Interior Camera (1st Person):** Positioned at the driver's seat inside the bus, looking forward through the windshield. Supports mouse-look for free head rotation.
3. **Free Camera:** Independent FPS-style camera with pitch/yaw from mouse input, WASD-style movement via arrow keys, and orbit mode around the bus (hold F).

Pseudocode: Camera View Matrix

Listing 3: Camera View Matrix Selection

```

1 FUNCTION getViewMatrix():
2     busRenderPos = busPosition + hoverHeight + bobOffset +
3         altitude
4
5     IF cameraMode == CHASE:
6         forward = getBusForward()
7         cameraPos = busRenderPos - forward * 18 + vec3(0, 5, 0)
8         lookTarget = busRenderPos + vec3(0, 1.5, 0) - forward * 2
9         RETURN myLookAt(cameraPos, lookTarget, UP)
10
11    IF cameraMode == INTERIOR:
12        cameraPos = busRenderPos + forward * (-3) + vec3(0, 1, 0)
13        + right * (-0.6)
14        lookDir = computeFromPitchYaw(cameraYaw, cameraPitch)
15        RETURN myLookAt(cameraPos, cameraPos + lookDir, UP)

```

```

16 // FREE CAMERA
17 front = computeFromPitchYaw(cameraYaw, cameraPitch)
18 up = computeUp(front, cameraRoll)
19 RETURN myLookAt(cameraPos, cameraPos + front, up)

```

5.5 Lighting and Shading

The project implements the Phong lighting model with three light types, computed in the fragment shader:

- **Directional Light:** Simulates sunlight with direction $(-0.2, -1.0, -0.3)$.
- **Point Lights (4):** Positioned around the bus, each with distinct color (red, green, blue, white) and attenuation parameters.
- **Spotlight:** Follows the camera direction with a 12.5° cutoff cone.

Each light computes three components:

$$I_{ambient} = k_a \cdot L_a \cdot C_{obj} \quad (1)$$

$$I_{diffuse} = k_d \cdot L_d \cdot \max(\hat{n} \cdot \hat{l}, 0) \cdot C_{obj} \quad (2)$$

$$I_{specular} = k_s \cdot L_s \cdot (\max(\hat{r} \cdot \hat{v}, 0))^{shininess} \quad (3)$$

where $\hat{n}$ is the surface normal, $\hat{l}$ is the light direction, $\hat{r}$ is the reflection vector, $\hat{v}$ is the view direction, and C_{obj} is the object color.

The project also supports Gouraud shading (vertex-level Phong) when `textureMode == 2`, where all lighting calculations are performed in the vertex shader and interpolated across fragments.

5.6 Texture Mapping

The project implements five texture modes:

0. **Off:** No texture, pure object color with Phong lighting.
1. **Pure Texture:** Texture color replaces object color entirely.
2. **Vertex-Blended (Gouraud):** Texture modulated by vertex-interpolated lighting.
3. **Fragment-Blended (Phong):** Texture modulated by per-fragment Phong lighting.
4. **Multi-Texture Blend:** Two textures blended spatially based on world position along a configurable axis, with smooth transition.

Textures are loaded using the `stb_image` library with configurable wrap modes (`GL_REPEAT`, `GL_CLAMP_TO_EDGE`, `GL_MIRRORED_REPEAT`) and filter modes (`GL_LINEAR`, `GL_NEAREST`). UV tiling is controlled via a `texScale` uniform for proportional texture mapping.

Pseudocode: Texture Loading

Listing 4: Texture Loading

```

1 FUNCTION loadTexture(path, wrapMode, filterMode):
2     data = stbi_load(path, &width, &height, &channels, 3)
3     IF width or height > MAX_TEXTURE_DIM:
4         data = downscale(data, MAX_TEXTURE_DIM)
5
6     glGenTextures(1, &textureID)
7     glBindTexture(GL_TEXTURE_2D, textureID)
8     glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, width, height,
9                 0, GL_RGB, GL_UNSIGNED_BYTE, data)
10    glGenerateMipmap(GL_TEXTURE_2D)
11    setWrapMode(wrapMode)
12    setFilterMode(GL_LINEAR_MIPMAP_LINEAR, filterMode)
13    RETURN textureID

```

5.7 Bezier, Spline, and Ruled Surfaces

Three types of parametric curve-based surfaces are used to create decorative city objects:

- **Bezier Surface of Revolution:** A cubic Bezier curve defines a profile that is revolved around the Y-axis. Used for decorative vases placed along road edges. The Bezier curve is evaluated using De Casteljau’s algorithm.
- **Catmull-Rom Spline Surface of Revolution:** A Catmull-Rom spline provides C^1 continuity through control points. Revolved around Y-axis to form smooth street lamp posts.
- **Ruled Surface:** A surface formed by linearly interpolating between two space curves. Used for canopy/awning structures at bus stop shelters.

5.8 Menger Sponge Fractal

The Menger Sponge is a 3D self-similar fractal. At each iteration, a cube is divided into $3 \times 3 \times 3 = 27$ sub-cubes and the 7 “axis-cross” cubes (face-centers and true center) are removed, leaving 20. After 3 iterations, $20^3 = 8000$ sub-cubes remain.

The fractal is **baked once at startup** into a list of (offset, size) pairs and rendered via GPU instancing in a single draw call per sponge. Sponges float above the road as collectible game elements.

Pseudocode: Menger Sponge Construction

Listing 5: Menger Sponge Fractal

```

1 FUNCTION buildMengerSponge(iterations):
2     current = [{offset: (0,0,0), size: 1.0}]
3
4     FOR it = 0 TO iterations-1:
5         next = empty list
6         FOR each cube c in current:
7             s = c.size / 3
8             FOR x = 0 TO 2:
9                 FOR y = 0 TO 2:

```

```

10         FOR z = 0 TO 2:
11             centersOnAxis = count(x==1, y==1, z==1)
12             IF centersOnAxis <= 1: // Keep this sub-cube
13                 offset = c.offset + vec3((x-1)*s, (y-1)*s, (z
14                     -1)*s)
15                 next.add({offset, s})
16             current = next
17
18 // Upload instance data to GPU for instanced rendering
uploadInstanceBuffer(current)

```

5.9 Fractal Forest (GPU-Instanced)

The fractal forest uses recursive branching to create realistic trees. At startup, all branch transforms and leaf transforms are baked into two instance buffers. At render time, the entire forest tile is drawn with just two instanced draw calls (one for branches, one for leaves).

- **Branches:** Low-poly, flat-shaded cylinders with asymmetric radii for ragged bark appearance. Textured with bark texture.
- **Leaves:** Crossed billboard quads (two perpendicular planes) with alpha-cutout leaf texture. The fragment shader discards transparent pixels.
- **Recursion:** Each branch spawns 3 child branches with randomized yaw, pitch, and scale. Depth = 5 yields 364 branches per tree.

Pseudocode: Fractal Tree Branching

Listing 6: Recursive Fractal Tree

```

1 FUNCTION bakeFractalBranch(branches, leaves, base, length,
2                             radius, depth, seed):
3     // Add this branch's transform
4     m = translate(base, vec3(0, length/2, 0))
5     m = scale(m, vec3(radius, length, radius))
6     branches.add(m)
7
8     tip = translate(base, vec3(0, length, 0))
9
10    IF depth <= 0:
11        // Add leaf cluster at branch tip
12        FOR k = 0 TO 11:
13            leafTransform = tip * randomOffset * randomRotation
14                            * scale(leafSize)
15            leaves.add(leafTransform)
16        RETURN
17
18    // Spawn 3 child branches
19    FOR i = 0 TO 2:
20        yaw = i * 120 + randomJitter
21        pitch = 22 + random * 22

```

```

22     child = rotate(tip, yaw, Y) * rotate(pitch, X)
23     bakeFractalBranch(branches, leaves, child,
24                       length*0.68, radius*0.62, depth-1, seed)

```

5.10 Collision Detection

AABB (Axis-Aligned Bounding Box) collision detection is used to prevent the bus from passing through buildings and other solid objects. Each frame:

1. All visible buildings and objects register their AABB into a collision list.
2. Before moving the bus, the new position is tested against all collision boxes.
3. If a collision is detected, the bus bounces back and the player loses points.

5.11 Gamification and Scoring

The scoring system involves:

- **Ring Checkpoints:** Flying through ring shapes awards +5 (circle torus) or +3 (polygon rings). Five shape variants exist: circle, hexagon, triangle, square, pentagon.
- **Menger Sponge Collectibles:** Flying through a sponge awards +15 points.
- **Building Collisions:** Hitting a building penalizes -2 points with a cooldown timer.
- **HUD Display:** Score is rendered using a custom bitmap font (3×5 pixel glyphs) drawn with cubes in screen-space orthographic projection. Score changes trigger visual feedback (color flash and bounce animation).

5.12 Skybox Rendering

A cubemap skybox is loaded from 6 individual face images and rendered as a unit cube at infinite distance. The view matrix translation is removed so the skybox stays centered on the camera. Depth function is set to `GL_LEQUAL` to pass the depth test at $z = 1.0$.

5.13 Rendering Pipeline

The project uses modern OpenGL rendering techniques:

- **Vertex Buffer Objects (VBOs):** Store vertex data including positions, normals, and texture coordinates.
- **Vertex Array Objects (VAOs):** Store the VBO configuration for efficient state management.
- **GPU Instancing:** Used for Menger Sponge and fractal forest rendering. Per-instance data (offset/scale or full mat4 transforms) is uploaded once and drawn via `glDrawArraysInstanced`.
- **Transformation Pipeline:** Model → View → Projection matrices are combined in the vertex shader. Per-instance transforms are applied before the model matrix.

6 Conclusion

This project successfully demonstrates the capabilities of Modern OpenGL 3.3 in creating an interactive 3D simulation with a rich feature set. The hover bus simulation integrates:

- **Hierarchical modeling** using primitive shapes (Cube, Cylinder, Torus, Sphere, Cone) built from parametric triangle representations.
- **Multiple camera systems** including chase, interior, and free cameras with a custom lookAt implementation.
- **Advanced lighting** with Phong and Gouraud shading models, supporting directional, point, and spot lights with per-component toggles.
- **Comprehensive texture mapping** with five modes, configurable wrap/filter, multi-texture blending, and UV tiling.
- **Parametric curve surfaces** including Bezier, Catmull-Rom spline, and ruled surfaces of revolution.
- **3D fractals** including Menger Sponges and recursive fractal forests, both GPU-instanced for performance.
- **Procedural city generation** with infinite, deterministic building placement.
- **Gamification** with checkpoints, collectibles, collision detection, and real-time HUD scoring.
- **Skybox rendering** using cubemap textures.

The simulation provides a robust foundation for exploring complex 3D graphics scenarios and serves as a comprehensive demonstration of real-time rendering techniques, shader programming, and interactive 3D application development.

7 References

1. Learn OpenGL — <https://learnopengl.com>
2. OpenGL Mathematics (GLM) Library — <https://github.com/g-truc/glm>
3. stb_image — Single-file image loading library — <https://github.com/nothings/stb>
4. GLFW — Window and input management — <https://www.glfw.org>
5. Provided in the classroom